

Sex-Based Discrimination in Promotion from Associate Professor to Full Professor

DATA 557 Final Project

This interactive data story examines whether sex-based disparities exist in promotion from Associate Professor to Full Professor using historical faculty salary and rank data from a major university.

Coding Setup

Imports

```
In [1]: import numpy as np
import pandas as pd
```

Load Data

```
In [2]: DATA_FILE_PATH = "Data557_FinalProject_Dataset.txt"

salary_df = pd.read_csv(DATA_FILE_PATH,
                        sep=r"\s+",
                        na_values="NA")

salary_df
```

Out[2]:

	case	id	sex	deg	yrdeg	field	startyr	year	rank	admin	salary
0	1	1	F	Other	92	Other	95	95	Assist	0	6684.0
1	2	2	M	Other	91	Other	94	94	Assist	0	4743.0
2	3	2	M	Other	91	Other	94	95	Assist	0	4881.0
3	4	4	M	PhD	96	Other	95	95	Assist	0	4231.0
4	5	6	M	PhD	66	Other	91	91	Full	1	11182.0
...	...	...	...	...	...	...	...	...	...	...	...
19787	19788	1770	M	Other	51	Other	64	91	Full	0	5318.0
19788	19789	1770	M	Other	51	Other	64	92	Full	0	5472.0
19789	19790	1770	M	Other	51	Other	64	93	Full	0	5551.0
19790	19791	1770	M	Other	51	Other	64	94	Full	0	5551.0
19791	19792	1770	M	Other	51	Other	64	95	Full	0	5712.0

19792 rows × 11 columns

Data Cleaning and Initial Validation

Before beginning the analysis, we load the dataset and perform basic data cleaning and validation checks.

The goal of this section is to:

- verify the dataset structure
- confirm variable types
- identify missing values
- check that key variables contain expected values
- ensure that each faculty member may appear across multiple years

Inspect Variable Types

This confirms:

- numeric variables
- categorical/string variables (usually listed as object)
- missing values

In [3]: `salary_df.info()`

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 19792 entries, 0 to 19791
Data columns (total 11 columns):
 #   Column      Non-Null Count  Dtype  
---  -
 0   case        19792 non-null  int64  
 1   id           19792 non-null  int64  
 2   sex          19792 non-null  object  
 3   deg          19792 non-null  object  
 4   yrdeg        19792 non-null  int64  
 5   field        19792 non-null  object  
 6   startyr      19792 non-null  int64  
 7   year         19792 non-null  int64  
 8   rank         19788 non-null  object  
 9   admin        19792 non-null  int64  
10  salary       19792 non-null  float64 
dtypes: float64(1), int64(6), object(4)
memory usage: 1.7+ MB
```

Numeric variable summary

We compute summary statistics for all numeric variables to verify that values fall within reasonable ranges and to check for potential data issues such as impossible values or unexpected outliers.

Summary statistics for numeric variables

In [4]: `numeric_summary = salary_df.describe()`
`numeric_summary`

Out[4]:

	case	id	yrdeg	startyr	year	admin	salary
count	19792.0000	19792.000000	19792.000000	19792.000000	19792.000000	19792.000000	19792.000000
mean	9896.5000	883.503739	72.106760	76.072201	87.432346	0.105042	4721.823453
std	5713.6026	505.710683	8.504135	8.951341	5.556229	0.306616	1986.705277
min	1.0000	1.000000	48.000000	48.000000	76.000000	0.000000	1200.000000
25%	4948.7500	461.000000	67.000000	69.000000	83.000000	0.000000	3287.000000
50%	9896.5000	873.000000	72.000000	76.000000	88.000000	0.000000	4353.000000
75%	14844.2500	1315.000000	78.000000	83.000000	92.000000	0.000000	5794.000000
max	19792.0000	1770.000000	96.000000	95.000000	95.000000	1.000000	14464.000000

Check Unique values for categorical variables

```
In [5]: print("Sex values:", salary_df["sex"].unique())
print("Rank values:", salary_df["rank"].unique())
print("Degree values:", salary_df["deg"].unique())
print("Field values:", salary_df["field"].unique())
```

Sex values: ['F' 'M']

Rank values: ['Assist' 'Full' 'Assoc' nan]

Degree values: ['Other' 'PhD' 'Prof']

Field values: ['Other' 'Arts' 'Prof']

Data Cleaning

```
In [6]: # Clean column names just in case there is whitespace
salary_df.columns = salary_df.columns.str.strip()
```

Check for Missing Values

```
In [7]: print(salary_df.isna().sum())
print(salary_df["rank"].value_counts(dropna=False))
print(salary_df["sex"].value_counts(dropna=False))
```

```
case      0
id         0
sex        0
deg         0
yrdeg      0
field      0
startyr    0
year       0
rank       4
admin      0
salary     0
dtype: int64
rank
Full      9211
Assoc     6529
Assist    4048
NaN         4
Name: count, dtype: int64
sex
M      15866
F       3926
Name: count, dtype: int64
```

Drop rows with missing rank

```
In [8]: rows_before = salary_df.shape[0]

salary_clean_df = salary_df.dropna(subset=["rank"]).copy()

rows_after = salary_clean_df.shape[0]

print("Rows before:", rows_before)
print("Rows after dropping missing rank:", rows_after)
print("Rows dropped:", rows_before - rows_after)

salary_clean_df["rank"].value_counts()
```

```
Rows before: 19792
Rows after dropping missing rank: 19788
Rows dropped: 4
```

```
Out[8]: rank
Full      9211
Assoc     6529
Assist    4048
Name: count, dtype: int64
```

Longitudinal structure check

The dataset is expected to have a **longitudinal (person-year) structure**, meaning that each faculty member may appear in multiple rows corresponding to different years of observation.

We verify this structure by confirming that faculty identifiers appear multiple times in the dataset and that observations span multiple years for each individual. This ensures that the data can be used to reconstruct career trajectories and identify promotion events over time.

Check how many entries per faculty

```
In [9]: rows_per_faculty = (salary_clean_df
                             .groupby("id")
                             .size())

print("Summary of number of rows per faculty:")
print(rows_per_faculty.describe())
```

```
Summary of number of rows per faculty:
count    1597.000000
mean      12.390733
std        6.718013
min         1.000000
25%         6.000000
50%        13.000000
75%        20.000000
max        20.000000
dtype: float64
```

Check how many faculty appear only once

```
In [10]: single_year_faculty = (rows_per_faculty == 1).sum()
multi_year_faculty = (rows_per_faculty > 1).sum()

print("Faculty with 1 year:", single_year_faculty)
print("Faculty with multiple years:", multi_year_faculty)
```

Faculty with 1 year: 72
Faculty with multiple years: 1525

Removing single-year faculty

Because the dataset is structured as person-year observations, faculty members with only one recorded year cannot be used to reconstruct career trajectories or identify promotion events.

We therefore remove faculty members who appear in the dataset for only a single year.

Identify & Remove faculty appearing only once

```
In [11]: single_year_ids = rows_per_faculty[rows_per_faculty == 1].index

salary_clean_df = salary_clean_df[~salary_clean_df["id"].isin(single_year_ids)]

print("Faculty removed:", len(single_year_ids))
print("Remaining rows:", salary_clean_df.shape[0])
print("Remaining faculty:", salary_clean_df["id"].nunique())
```

Faculty removed: 72
Remaining rows: 19716
Remaining faculty: 1525

Check that the year varies within a faculty member

```
In [12]: years_per_faculty = (salary_clean_df
                               .groupby("id")["year"]
                               .nunique())
```

```
print(years_per_faculty.describe())
```

```
count    1525.000000
mean      12.928525
std       6.390980
min       2.000000
25%       7.000000
50%      13.000000
75%      20.000000
max      20.000000
Name: year, dtype: float64
```

Rank trajectory validation

Because this project studies promotion from Associate Professor to Full Professor, we next validate whether faculty rank histories are consistent with longitudinal career progression.

This check helps identify cases where rank trajectories may be ambiguous or unsuitable for promotion analysis, such as faculty who are observed only at the Full Professor rank or cases where observed rank changes move backward over time.

Sort Data and then check for rank patterns by faculty member

```
In [13]: salary_clean_df = salary_clean_df.sort_values(["id", "year"]).copy()

rank_history_df = (salary_clean_df
                   .groupby("id")["rank"]
                   .agg(lambda rank_values: tuple(rank_values))
                   .reset_index(name="rank_history"))

rank_history_df.head()
```


Out[13]:

	id	rank_history
0	2	(Assist, Assist)
1	6	(Full, Full, Full, Full, Full)
2	7	(Assist, Assist, Assist, Assoc, Assoc, Assoc, ...)
3	9	(Assist, Assist, Assist, Assoc, Assoc, Assoc, ...)
4	10	(Assoc, Assoc, Assoc, Assoc, Assoc, Assoc, Ass...

Check unique rank sets by faculty member

```
In [14]: rank_set_df = (salary_clean_df
    .groupby("id")["rank"]
    .agg(lambda rank_values: tuple(sorted(set(rank_values))))
    .reset_index(name="rank_set"))

rank_set_counts = (rank_set_df["rank_set"]
    .value_counts()
    .reset_index())

rank_set_counts.columns = ["rank_set", "faculty_count"]
```

Identify and remove any faculty with backward rank movement

```
In [15]: # Encode rank order for trajectory validation
RANK_ORDER = {"Assist": 1,
              "Assoc": 2,
              "Full": 3}

salary_clean_df["rank_order"] = salary_clean_df["rank"].map(RANK_ORDER)

# Identify faculty with backward rank movement
def has_backward_rank_movement(rank_order_series):
    rank_order_list = list(rank_order_series)
    return any(
        current_rank < previous_rank
```

```

    for previous_rank, current_rank in zip(rank_order_list[:-1], rank_order_list[1:]))

backward_rank_df = (salary_clean_df
                    .groupby("id")["rank_order"]
                    .apply(has_backward_rank_movement)
                    .reset_index(name="has_backward_movement"))

backward_rank_cases = backward_rank_df[backward_rank_df["has_backward_movement"]]

print("Faculty with backward rank movement:", backward_rank_cases.shape[0])
backward_rank_cases.head()

```

Faculty with backward rank movement: 1

Out[15]:

	id	has_backward_movement
1222	1403	True

Inspect backward rank case

In [16]:

```

backward_rank_ids = backward_rank_cases["id"].tolist()
salary_clean_df[salary_clean_df["id"].isin(backward_rank_ids)].sort_values(["id", "year"])

```

Out[16]:

	case	id	sex	deg	yrdeg	field	startyr	year	rank	admin	salary	rank_order
15825	15826	1403	M	PhD	71	Other	71	76	Assoc	0	1349.0	2
15826	15827	1403	M	PhD	71	Other	71	77	Assist	0	1828.0	1
15827	15828	1403	M	PhD	71	Other	71	78	Assist	0	1942.0	1
15828	15829	1403	M	PhD	71	Other	71	79	Assist	0	2202.0	1
15829	15830	1403	M	PhD	71	Other	71	80	Assist	0	2376.0	1
15830	15831	1403	M	PhD	71	Other	71	81	Assist	0	2798.0	1
15831	15832	1403	M	PhD	71	Other	71	82	Assoc	0	3048.0	2
15832	15833	1403	M	PhD	71	Other	71	83	Assoc	0	3216.0	2
15833	15834	1403	M	PhD	71	Other	71	84	Assoc	0	3484.0	2
15834	15835	1403	M	PhD	71	Other	71	85	Assoc	0	3484.0	2
15835	15836	1403	M	PhD	71	Other	71	86	Assoc	0	4081.0	2
15836	15837	1403	M	PhD	71	Other	71	87	Full	0	4410.0	3
15837	15838	1403	M	PhD	71	Other	71	88	Full	0	4538.0	3
15838	15839	1403	M	PhD	71	Other	71	89	Full	0	4855.0	3
15839	15840	1403	M	PhD	71	Other	71	90	Full	0	5082.0	3
15840	15841	1403	M	PhD	71	Other	71	91	Full	0	5318.0	3
15841	15842	1403	M	PhD	71	Other	71	92	Full	0	5631.0	3
15842	15843	1403	M	PhD	71	Other	71	93	Full	0	5794.0	3
15843	15844	1403	M	PhD	71	Other	71	94	Full	0	5794.0	3
15844	15845	1403	M	PhD	71	Other	71	95	Full	0	5962.0	3

Rank trajectory anomaly

During the rank trajectory validation step, one faculty member was observed to move from Associate Professor to Assistant Professor before later returning to Associate Professor and eventually being promoted to Full Professor. This represents a backward rank transition that is inconsistent with the typical academic promotion pathway and likely reflects either a data recording anomaly or an unusual change in appointment status.

Because this individual is later observed transitioning from Associate Professor to Full Professor, the promotion event relevant to this analysis is still clearly defined. However, many of the subsequent summary measures we are going to make, such as first year as associate, might not be accurate, so we have chosen to remove this faculty from the dataset

```
In [17]: ANOMALOUS_FACULTY_ID = 1403

salary_clean_df = salary_clean_df[
    salary_clean_df["id"] != ANOMALOUS_FACULTY_ID].copy()

print("Remaining rows:", salary_clean_df.shape[0])
print("Remaining faculty:", salary_clean_df["id"].nunique())
```

```
Remaining rows: 19696
Remaining faculty: 1524
```

Save cleaned dataset

```
In [18]: salary_clean_df.to_csv("cleaned_longitudinal_dataset.csv", index=False)
```